

Продвинутый DDL. Основы проектирования баз данных.

Лекция 3

План

- Ограничения целостности
- Модификация таблиц
- Проектирование баз данных

Создание таблиц (напоминание)

Упрощенная форма запроса для создания таблиц выглядит так

```
CREATE TABLE схема.имя-таблицы(  
    имя-поля тип-данных [ограничения-целостности],  
    ...  
    имя-поля тип-данных [ограничения-целостности],  
    [ограничения-целостности],  
    [первичный-ключ],  
    [внешний-ключ]  
);
```

Ограничение CHECK

Объявление: **[CONSTRAINT имя] CHECK** проверка. При пропуске имени, генерируется имя по умолчанию.

Место объявления: **атрибут или таблица**

```
CREATE TABLE readers (  
    ...  
    status TEXT CHECK (status IN ('active', 'inactive')),  
    passport_sn VARCHAR(4) NOT NULL CONSTRAINT sn_check CHECK  
(length(passport_sn) = 4),  
    passport_no VARCHAR(6) NOT NULL CHECK (length(passport_no) = 6),  
    ...
```



генерируется имя
`readers_passport_no_check`

В psql `\d readers` выведет:

```
...  
Check constraints:  
"readers_passport_no_check" CHECK (length(passport_no::text) = 6)  
"sn_check" CHECK (length(passport_sn::text) = 4)
```

Ограничения CHECK (уровень таблицы)

Предыдущий запрос эквивалентен следующему:

```
CREATE TABLE readers (  
    ...  
    passport_sn VARCHAR(4) NOT NULL,  
    passport_no VARCHAR(6) NOT NULL,  
    ...  
    CONSTRAINT sn_check CHECK (length(passport_sn) = 4) ,  
    CHECK (length(passport_no) = 6) ,  
    ...
```

Ограничение CHECK (связь атрибутов)

```
CREATE TABLE status_demo (  
    id serial primary key,  
    status text not null check( status in ('created', 'updated')),  
    updated_at timestamptz DEFAULT NULL,  
    constraint not_null_updated  
        check ( status != 'updated' OR updated_at IS NOT NULL)  
);
```

Ограничение NOT NULL

Функционально эквивалентно **CHECK (column_name IS NOT NULL)**, но более принятым считается NOT NULL уровня атрибута

Ограничение UNIQUE

```
CREATE TABLE readers (  
    id serial PRIMARY KEY,  
    card_no VARCHAR(20) UNIQUE, ← Уровень атрибута  
    passport_sn VARCHAR(4) NOT NULL,  
    passport_no VARCHAR(6) NOT NULL,  
    ...  
    UNIQUE (passport_sn, passport_no) ← Уровень таблицы  
);
```

Также можно давать имя

```
CONSTRAINT unique_passsport UNIQUE (passport_sn, passport_no)
```


Составной первичный ключ

```
CREATE TABLE readers (  
    passport_sn VARCHAR(4) NOT NULL,  
    passport_no VARCHAR(6) NOT NULL,  
    ...  
    PRIMARY KEY (passport_sn, passport_no)  
);
```

Внешний ключ

```
CREATE TABLE library.readers (  
    card_no VARCHAR(20) PRIMARY KEY,  
    ...  
  
CREATE TABLE book_loans (  
    id SERIAL PRIMARY KEY,  
    card_no VARCHAR(20) REFERENCES readers (card_no),  
    book_name text not null  
);
```

Если в целевой и ссылающейся таблице имена атрибутов совпадают, то можно опустить имя в целевой таблице

```
card_no VARCHAR(20) REFERENCES readers
```

Внешний ключ (множество атрибутов)

Допускаются ссылки на уникальный ключ, не являющийся первичным, а также на ключ, состоящий из двух и более атрибутов.

```
CREATE TABLE book_loans (  
    id SERIAL PRIMARY KEY,  
    passport_sn VARCHAR(4) NOT NULL,  
    passport_no VARCHAR(6) NOT NULL,  
    book_name text not null,  
    FOREIGN KEY (passport_sn, passport_no) REFERENCES readers(passport_sn,  
passport_no)  
);
```

типы данных и количество атрибутов должны совпадать

Также можно дать ограничению имя с помощью CONSTRAINT

```
CONSTRAINT passport_ref FOREIGN KEY (passport_sn, passport_no) REFERENCES  
readers(passport_sn, passport_no)
```

Внешний ключ (nullable атрибут)

```
CREATE TABLE book_loans (  
    id SERIAL PRIMARY KEY,  
    passport_sn VARCHAR(4), ← Теперь здесь может быть null  
    passport_no VARCHAR(6) NOT NULL,  
    book_name text not null,  
    FOREIGN KEY (passport_sn, passport_no) REFERENCES readers(passport_sn,  
passport_no)  
);
```

Возможна ли вставка?

```
INSERT INTO book_loans (passport_sn, passport_no, book_name)  
VALUES (NULL, 'sdfgh', 'test');
```

Да, возможна! Postgres пропускает проверку, если хотя бы одно значение null.

Политики обеспечения ссылочной целостности

Что произойдет, если попытаться удалить или обновить запись из целевой таблицы при существующей записи в ссылающейся таблице?

1. Запрет на удаление

```
card_no VARCHAR(20) REFERENCES readers (card_no) ON DELETE NO ACTION,  
card_no VARCHAR(20) REFERENCES readers (card_no) ON DELETE RESTRICT,
```

RESTRICT – немедленная проверка

NO ACTION (по-умолчанию) – откладывает проверку на конец транзакции

2. Удаление связанных строк в ссылающейся таблице

```
card_no VARCHAR(20) REFERENCES readers (card_no) ON DELETE CASCADE,
```

3. Присваивание NULL атрибуту внешнего ключа

```
card_no VARCHAR(20) REFERENCES readers (card_no) ON DELETE SET NULL,
```

4. Присваивание атрибуту внешнего ключа значения по-умолчанию

```
card_no VARCHAR(20) REFERENCES readers (card_no) ON DELETE SET DEFAULT,
```

ALTER TABLE

Изменение существующей таблицы

Команда **ALTER TABLE** позволяет выполнить (но не ограничивается):

- переименование таблицы
- перемещение в другую схему
- добавление/удаление/переименование столбцов
- изменение типа данных столбцов
- установка/удаление значения DEFAULT
- установка/удаление ограничений NOT NULL, CHECK, UNIQUE, PRIMARY KEY, FOREIGN KEY
- и многое другое...

<https://postgrespro.ru/docs/postgresql/18/sql-altertable>

Переименование таблицы и перемещение в другую схему

`ALTER TABLE readers RENAME TO library_members;` — переименование таблицы

`ALTER TABLE readers SET SCHEMA public;` — перемещение в другую схему

При перемещении в схему:

- схема уже должна существовать
- у пользователя должны быть соответствующие права на обе схемы
- также перемещаются все связанные с таблицей ограничения, последовательности и индексы

Добавление, удаление, переименование столбцов

```
ALTER TABLE readers ADD COLUMN email TEXT,  
    ADD COLUMN address TEXT NOT NULL DEFAULT 'не указан';
```

 – добавление

```
ALTER TABLE readers RENAME COLUMN email TO mail;
```

 – переименование

```
ALTER TABLE readers DROP COLUMN mail, DROP COLUMN address;
```

 – удаление

Проверки на существование и несуществование:

```
ALTER TABLE readers ADD COLUMN IF NOT EXISTS email TEXT;  
ALTER TABLE readers DROP COLUMN IF EXISTS email;
```

Также можно добавить поле с CHECK ограничениями

```
ALTER TABLE readers ADD COLUMN reader_category TEXT  
    CHECK (reader_category IN ('child', 'student', 'adult'));
```

Установка и удаление значения DEFAULT

`ALTER TABLE readers ALTER COLUMN reader_category SET DEFAULT 'adult';` – установка DEFAULT значения (добавление или изменение)

`ALTER TABLE readers ALTER COLUMN reader_category DROP DEFAULT;` – удаление DEFAULT значения

Данные команды влияют только на новые строки, существующие не меняются

Установка и удаление ограничений

Установка и удаление NOT NULL

```
ALTER TABLE readers ALTER COLUMN email SET NOT NULL;  
ALTER TABLE readers ALTER COLUMN email DROP NOT NULL;
```

Установка и удаление CHECK

```
ALTER TABLE readers ADD CONSTRAINT check_email_format  
    CHECK (email LIKE '%@%');  
ALTER TABLE readers DROP CONSTRAINT check_email_format;
```

Установка и удаление UNIQUE

```
ALTER TABLE readers ADD CONSTRAINT unique_email UNIQUE (email);  
ALTER TABLE readers DROP CONSTRAINT unique_email;
```

Установка и удаление ключей

Установка и удаление primary key

```
ALTER TABLE book_loans DROP CONSTRAINT book_loans_pkey;  
ALTER TABLE book_loans ADD PRIMARY KEY (card_no, book_name);
```

Добавление и удаление foreign key

```
ALTER TABLE book_loans DROP CONSTRAINT book_loans_card_no_fkey;  
ALTER TABLE book_loans ADD CONSTRAINT book_loans_card_no_fkey  
    FOREIGN KEY (card_no) REFERENCES readers;
```

Можно опустить имя для добавления

```
ALTER TABLE book_loans ADD FOREIGN KEY (card_no) REFERENCES readers;
```

Список ограничений можно узнать с помощью \d в psql, либо выполнив

```
SELECT conname, contype FROM pg_constraint  
    WHERE conrelid = 'book_loans'::regclass;
```

Изменение типа данных столбцов

```
ALTER TABLE readers ALTER COLUMN card_no TYPE TEXT;
```

```
ALTER TABLE readers ALTER COLUMN birth_date TYPE TIMESTAMP;
```

– автоматическое приведение типов из одной группы (varchar -> text, date -> timestamp)

```
ALTER TABLE readers ALTER COLUMN birth_date TYPE TEXT USING birth_date::TEXT;
```

– явное приведение, используя USING

```
ALTER TABLE readers ALTER COLUMN reader_category SET DATA TYPE INTEGER  
USING ( CASE WHEN reader_category = 'child' THEN 1  
           WHEN reader_category = 'student' THEN 2  
           ELSE 3  
        END);
```

– преобразование с маппингом

Проектирование баз данных

Основные определения

Проектирование БД – процесс создания схемы базы данных и определения необходимых ограничений целостности для выполнения поставленных задач.

Основные задачи:

- Хранение в БД всей необходимой информации
- Обеспечение целостности данных (исключение противоречий, предотвращение потерь и т.д.)
- Выполнение нефункциональных требований (задержки, requests per second (RPS), доступность)

Этапы проектирования БД

1. Анализ предметной области

- a. изучение документации
- b. общение со специалистами
- c. изучение нефункциональных требований

2. Логическое проектирование

- a. определение сущностей и их атрибутов
- b. построение связей между сущностями
- c. нормализация данных
- d. построение Entity Relationship (ER) диаграммы

3. Физическое проектирование

- a. Выбор СУБД (нам не нужен:))
- b. Конвертация ER диаграммы в DDL
- c. денормализация данных (если нужно)
- d. определение индексных структур и способов хранения (репликация и шардирование)

Нормализация данных

Это способ организации данных в БД, при котором отсутствуют повторяющиеся данные. Нормализация обеспечивает:

- уменьшение объема базы данных
- упрощение поиска
- уменьшение вероятности ошибок и аномалий

Нормализация обеспечивается “поэтапным” устранением дублирования. Каждый этап можно представить сводом правил, называемы **нормальной формой**.

Первая нормальная форма (1НФ)

- в отношении нет повторяющихся групп атрибутов
- все атрибуты атомарны

Повторяющиеся группы

Courseld	Lecturer	Phone (1)	Phone (2)
1	Корнеев Г. А.		111-11-11
2	Киракозов А. Х.	222-22-22	333-33-33
3	Кудряшов Б. Д.	444-44-44	555-55-55
4	Сегаль А. С.	666-66-66	

Неатомарные атрибуты

Courseld	Lecturer	Phones
1	Корнеев Г. А.	111-11-11
2	Киракозов А. Х.	222-22-22
		333-33-33
3	Кудряшов Б. Д.	444-44-44
		555-55-55
4	Сегаль А. С.	666-66-66

Приведение к 1НФ

Courseld	Lecturer	Phone
1	Корнеев Г. А.	111-11-11
2	Киракозов А. Х.	222-22-22
2	Киракозов А. Х.	333-33-33
3	Кудряшов Б. Д.	444-44-44
3	Кудряшов Б. Д.	555-55-55
4	Сегаль А. С.	666-66-66

Вторая нормальная форма (2НФ)

- отношение в 1НФ
- есть первичный ключ
- все неключевые атрибуты функционально зависят от ключа целиком, но не от его части

Courseld	Year	Lecturer	Exam
1	2020	Корнеев Г. А.	yes
2	2019	Киракозов А. Х.	no
2	2020	Киракозов А. Х.	no
3	2019	Левина А. Б.	yes
3	2020	Чепурной А. И.	yes

Здесь ключ (Courseld, Year)

При этом:

- Courseld -> Exam
- Courseld, Year -> Lecturer

Exam зависит от части ключа

Приведение к 2НФ

Courseld	Year	Lecturer
1	2020	Корнеев Г. А.
2	2019	Киракозов А. Х.
2	2020	Киракозов А. Х.
3	2019	Левина А. Б.
3	2020	Чепурной А. И.

Courseld	Exam
1	yes
2	no
3	yes

Третья нормальная форма 3НФ

- отношение в 2НФ
- неключевые атрибуты непосредственно (нетранзитивно) функционально зависят от ключей

Courseld	Year	Lecturer	Phone
1	2020	Корнеев Г. А.	111-11-11
2	2019	Киракозов А. Х.	222-22-22
2	2020	Киракозов А. Х.	222-22-22
3	2019	Левина А. Б.	333-33-33
3	2020	Чепурной А. И.	444-44-44

Ключ (Courseld, Year)

- Courseld, Year -> Lecturer
- Lecturer -> Phone – транзитивная зависимость

Приведение к 3НФ

Courseld	Year	Lecturer
1	2020	Корнеев Г. А.
2	2019	Киракозов А. Х.
2	2020	Киракозов А. Х.
3	2019	Левина А. Б.
3	2020	Чепурной А. И.

Lecturer	Phone
Корнеев Г. А.	111-11-11
Киракозов А. Х.	222-22-22
Левина А. Б.	333-33-33
Чепурной А. И.	444-44-44

Логическое проектирование (терминология)

- **Предметная область (ПрО)** – ограниченная область реального мира. База данных - это модель (абстракция, упрощение) предметной области.
- **Сущность предметной области** – абстрактное понятие, которое объединяет группу реальных объектов из ПрО, имеющих одинаковый набор атрибутов и значение
- **Атрибуты сущности** – свойства, которыми описывается сущность
- **Связь** – осмысленная ассоциация между сущностями

Логическое проектирование (пример ПрО)

Библиотека – ограниченная область реального мира, включающая:

- процесс выдачи и возврата книг
- учет читателей и их читательских билетов
- каталог книг и авторов
- контроль сроков возврата и штрафов

База данных библиотеки - это упрощённая модель, которая **не учитывает**:

- физическое состояние книг
- расположение книг на полках
- температуру в читальном зале
- ЗП сотрудников

Логическое проектирование (пример сущность ПрО)

Читатель – абстрактное понятие, объединяющее всех реальных людей, которые:

- имеют читательский билет
- зарегистрированы в библиотеке
- имеют одинаковый набор характеристик: ФИО, паспорт, дата рождения

Издание книги – объединяет все физические экземпляры книг, изданные в определенный период времени конкретным издательством

Книга – обозначает конкретную книгу на полке

Выдача книги – сущность, которая фиксирует факт выдачи книги

Логическое проектирование (пример атрибут сущности)

Читатель:

- номер читательского билета (card_no)
- серия и номер паспорта
- имя, фамилия
- дата рождения

Книга:

- инвентарный номер
- название
- год издания
- ISBN
- количество страниц

Логическое проектирование (пример связь)

“Книга принадлежит изданию” – ассоциация между Издание и Книга

“Автор написал книгу” – ассоциация между Автор и Книга

“Читатель берет книгу” – ассоциация между сущностями Читатель и Книга через сущность-связку “Выдача книги”:

- читатель с билетом LIB-2024-0001 взял книгу с инв. номером 12345
- дата выдачи: 15 марта 2024
- срок возврата: 30 марта 2024

Сильные и слабые сущности

Сильная сущность – сущность, которая может существовать сама по себе

- Читатель
- Автор
- Издание книги

Слабая сущность – та, которая не может существовать без родительской (владеющей) сущности

- Книга (не существует без издания)
- Выдача книги (не существует без читателя и без книги)

С точки зрения реляционной модели, сильные сущности всегда имеют собственный РК, а слабые сущности имеют РК являющийся FK

Виды связей

- **Связь 1:1.** Одиночный экземпляр сущности одного класса связан с одиночным экземпляром сущности другого класса.
- **Связь 1:M.** Единый экземпляр сущности одного класса связан со многими экземплярами сущности другого класса.
- **Связь M:N.** Несколько экземпляров сущности одного класса связаны с несколькими экземплярами сущности другого класса.

Примеры:

- **Один** сотрудник библиотеки имеет **одно** рабочее место
- **Один** читатель может брать **много** книг
- **Одна** книга может иметь несколько авторов, **один** автор может написать несколько книг.

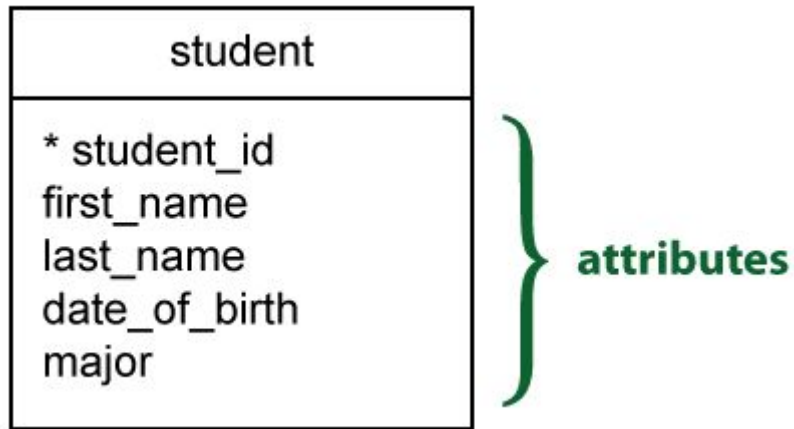
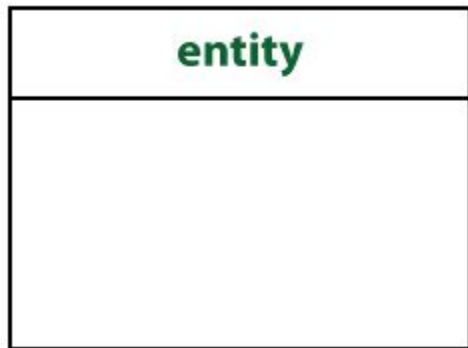
А где построить ER-диаграмму?

CASE-системы (Computer-Aided Software Engineering) – это методы и технологии, которые позволяют проектировать различные информационные системы (в частности, базы данных) и автоматизировать их создание.

Включают:

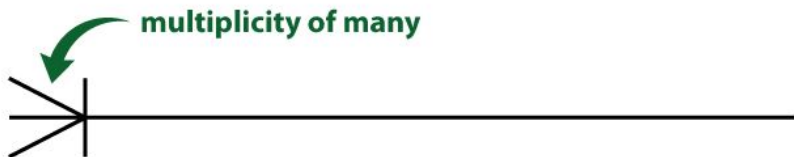
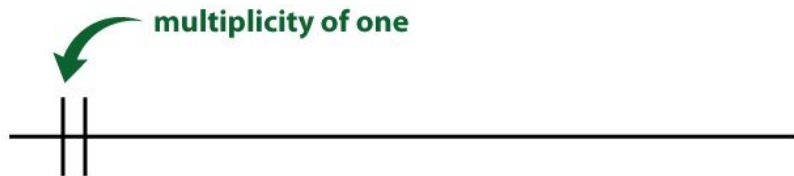
- Средства анализа
- Средства проектирования
- Средства визуализации
- Средства хранения артефактов

Построение ER-диаграммы (crow's foot notation)

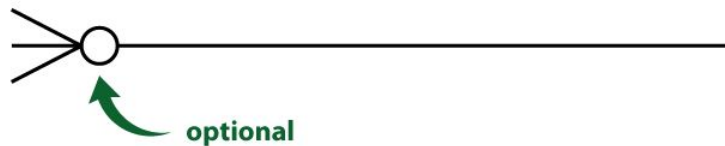


Построение ER-диаграммы (связи)

Множественность (| или >)



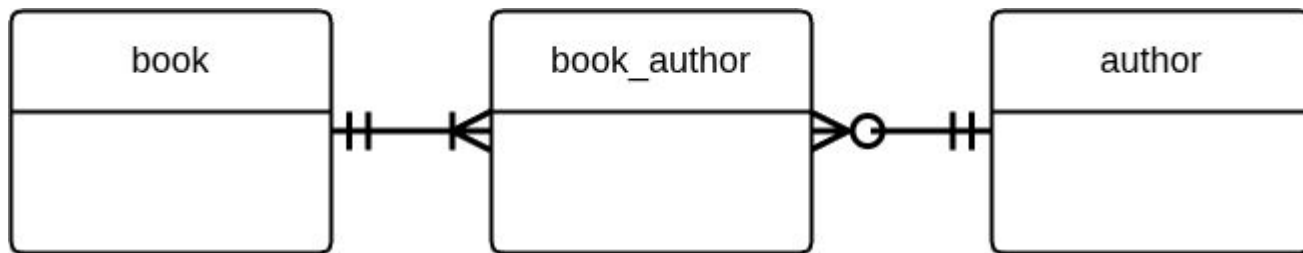
Обязательность (о или |)



Иногда множественность пропускают. В таком случае, считается, что множественность равна 1 (или |).

Оба конца важны при чтении ER-диаграммы!

Построение ER-диаграммы (читаем диаграмму)



Слева направо (у книги всегда есть хотя бы один автор):

сколько **book_authors** у одной **book**? один или много

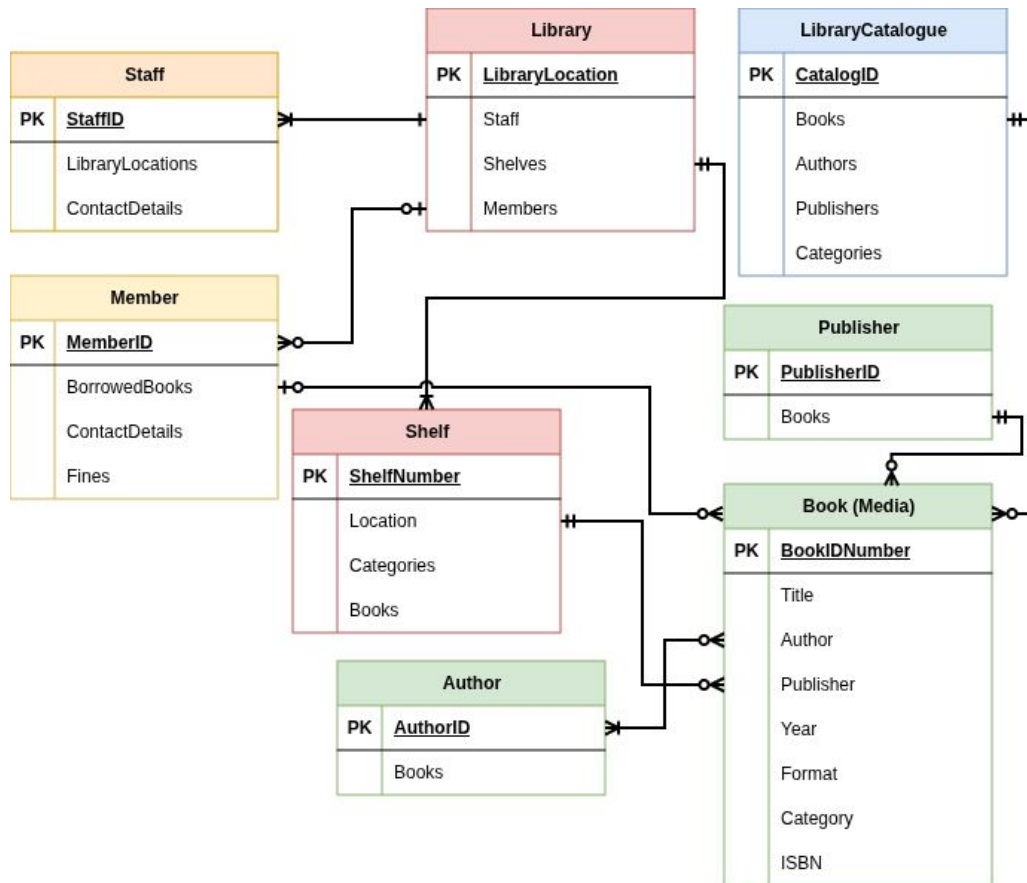
сколько **author** у одного **book_author**? ровно один

Справа налево (у автора может не быть книг, а может быть много):

сколько **book_authors** у одного **author**? ноль или много

сколько **book** у одного **book_author**? ровно один

Построение ER-диаграммы (пример)



Правила переноса ER-диаграммы в SQL

ER элемент	SQL реализация	Пример
Сущность	TABLE	CREATE TABLE readers
Атрибут	Колонка	first_name TEXT
Ключевой атрибут	PRIMARY KEY	card_no PRIMARY KEY
Связь 1:N	FK на стороне N	card_no REFERENCES readers
Связь 1:1	FK + UNIQUE или объединить	reader_id UNIQUE REFERENCES
Связь M:N	Промежуточная таблица	book_authors с 2 FK
Обязательность	NOT NULL	NOT NULL REFERENCES
Уникальность	UNIQUE	UNIQUE (passport_sn, passport_no)

Пара слов про связь 1:1

Два варианта реализации

- Одна таблица (пользователи и они же читатели)
- Две таблицы с FK + UNIQUE (отдельно таблицы для пользователей и читателей)

Используем второй вариант, когда

- Данные опциональны
- Разделение ответственности
- Большой набор атрибутов